

CKA: Storage

KodeKloud Complete Study Notes — Certified Kubernetes Administrator

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Storage Section Introduction

This guide explores essential Kubernetes storage concepts including persistent volumes, claims, access modes, and configuring applications for persistent storage.

Welcome to the Kubernetes Storage section! In this guide, we will explore essential storage concepts including persistent volumes (PV), persistent volume claims (PVC), and access modes. You'll also learn how to configure your applications for persistent storage in Kubernetes.

Kubernetes provides a versatile range of storage options that can adapt to different environments. This lesson specifically focuses on Kubernetes-native storage mechanisms, ensuring you understand the core concepts needed to integrate any third-party storage solutions with your cluster.

Make sure to review the Kubernetes documentation on storage for additional details and best practices.

Let's dive in and explore how to manage storage effectively in your Kubernetes deployments!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Storage in Docker

This guide explores advanced Docker storage concepts, including storage drivers, data management, and layered architecture for efficient image and container handling.

Welcome to this guide on advanced Docker storage concepts. In this article, we explore how Docker handles storage drivers, manages data on the host file system, and implements a layered architecture to build images and run containers efficiently.

When Docker is installed, it creates a folder structure at `/var/lib/docker` containing subdirectories such as `overlay2`, `containers`, `images`, and `volumes`. These directories store Docker images, container runtime data, and volumes. For instance, files associated with running containers reside in the `containers` folder, image files are stored under `images`, and any created volumes are kept in the `volumes` folder.

Docker Image Layers

Docker images are built using a layered architecture. Each instruction in a Dockerfile generates a new layer, containing only the modifications from the previous layer. Consider this Dockerfile for our first application:

```
```dockerfile theme={null}
```

## Dockerfile for Application 1

```
FROM ubuntu
```

```
RUN apt-get update && apt-get -y install python RUN pip install flask flask-mysql
```

```
COPY ./opt/source-code
```

```
ENTRYPOINT FLASK_APP=/opt/source-code/app.py flask run
```

You can build the image using:

```
```bash theme={null}
docker build Dockerfile -t mmumshad/my-custom-app
```

The layers are created in the following order:

1. The base Ubuntu image (approximately 120 MB).
2. A layer installing APT packages (around 300 MB).
3. A layer for Python package dependencies.
4. A layer adding the application source code.
5. A layer that sets the entry point.

Because each layer stores only the changes made in the previous one, Docker caches them for reuse in similar images. For example, a second application with a slight modification might use the following Dockerfile:

```
```dockerfile theme={null}
```

## Dockerfile2 for Application 2

---

```
FROM ubuntu RUN apt-get update && apt-get -y install python RUN pip install flask flask-mysql COPY app2.py /opt/source-code ENTRYPOINT FLASK_APP=/opt/source-code/app2.py flask run
```

Build the second image with:

```
```bash theme={null}
docker build Dockerfile2 -t mmumshad/my-custom-app-2
```

Since the first three layers (base image, APT packages, and Python dependencies) are identical, Docker reuses these cached layers and builds only the layers related to the new source code and entry point. This efficient reuse reduces build times and conserves disk space.

When application code changes (for example, modifying `app.py`), Docker leverages the cache for all unchanged layers and rebuilds only the layer with the new code.

Container Writable Layer and Copy-On-Write

Once an image is built, its layers remain immutable (read-only). Running a container from that image with the `docker run` command creates an additional writable layer on top. This layer captures any runtime modifications such as log files, temporary files, or changes to the application. For example:

```
```bash theme={null} docker run mmumshad/my-custom-app
```

If you log into the container and modify a file (say, creating `temp.txt`), Docker employs a copy-on-write

<Frame>

! [The image illustrates the "Copy-On-Write" concept, showing container layers with read-write access and

</Frame>

### ## Persistent Data with Volumes and Bind Mounts

The container's writable layer is ephemeral, meaning any data stored there is lost when the container is

#### ### Volume Mounts

Volumes are managed by Docker and stored under `/var/lib/docker/volumes`. Create and mount a volume with

```
```bash theme={null}
docker volume create data_volume
docker run -v data_volume:/var/lib/mysql mysql
```

If you run a container with a volume name that doesn't exist, Docker will automatically create it:

```
```bash theme={null} docker run -v data_volume2:/var/lib/mysql mysql
```

#### ### Bind Mounts

Bind mounts allow you to use a specific directory from the Docker host. For example, to use data from /d

```
```bash theme={null}
docker run -v /data/mysql:/var/lib/mysql mysql
```

Using the --mount Option

The `--mount` flag provides a more explicit syntax by requiring all parameters to be specified. The following command is equivalent to the bind mount example above:

```
bash theme={null} docker run \ --mount type=bind,source=/data/mysql,target=/var/lib/mysql \
mysql
```

Docker Storage Drivers

Docker's storage drivers manage everything from maintaining image layers to handling writable container layers with copy-on-write. Common storage drivers include AUFS, ZFS, BTRFS, Device Mapper, Overlay, and Overlay2. The selection of a storage driver depends on the host OS. For example, Ubuntu often uses AUFS by default, while Fedora or CentOS might prefer Device Mapper. Docker automatically selects the most appropriate driver for your system based on performance and stability factors.

Storage drivers

- AUFS
- ZFS
- BTRFS
- Device Mapper
- Overlay
- Overlay2

For more detailed information on these storage drivers, please refer to the [Docker documentation](#).

Summary

Docker's innovative approach to managing storage through image layers, copy-on-write, volumes, and storage drivers enables efficient container builds and resource usage. Understanding these concepts not only improves your workflow but also optimizes container performance and data persistence.

Thank you for reading, and happy containerizing!

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Volume Driver Plugins in Docker

This article discusses volume driver plugins in Docker for managing persistent storage across various third-party storage solutions.

In previous discussions, we explored storage drivers and their role in managing storage for images and containers. We also examined volumes and emphasized that if you need persistent storage, you must create volumes. It is important to note that volumes are not managed by storage drivers; instead, they are handled by volume driver plugins.

By default, Docker uses the local volume driver plugin. This plugin creates a volume on the Docker host and stores its data under the `/var/lib/docker/volumes` directory. However, many other volume driver plugins are available, enabling you to create volumes on third-party storage solutions such as Azure File Storage, Convoy, DigitalOcean Block Storage, Blocker, Google Compute Persistent Disks, ClusterFS, NetApp, Rex Ray, Portworx, and VMware vSphere Storage.

STORAGE DRIVERS

AUFS | ZFS | BTRFS | DEVICE MAPPER | OVERLAY

VOLUME DRIVERS

Local | Azure File Storage | Convoy |
DigitalOcean Block Storage | Flocker | gce-docker | GlusterFS
| NetApp | RexRay | Portworx | VMware vSpher

https://docs.docker.com/engine/extend/legacy_plugins/

When selecting a volume driver plugin, consider your storage infrastructure requirements. Plugins such as Rex Ray offer flexibility by supporting various storage providers.

For example, some volume drivers support multiple storage providers. The Rex Ray storage driver, for instance, can provision storage on AWS EBS, S3, EMC storage arrays (like Isilon and ScaleIO), Google Persistent Disk, or OpenStack Cinder. When running a Docker container, you can select a specific volume driver such as Rex Ray EBS to provision a volume from Amazon EBS. This approach ensures that your data remains safe in the cloud even if the container exits.

Below is an example of running a Docker container using a specific volume driver plugin:

```
bash theme={null} docker run -it \ --name mysql \ --volume-driver rexray/ebs \ --mount src=ebs-vol,target=/var/lib/mysql \ mysql
```

Later in the series, we will further explore how volumes are managed within Kubernetes.

For additional details, refer to the [Docker Documentation](#) and learn more about [persistent storage in containers](#).

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/llms.txt> Use this file to discover all available pages before exploring further.

Container Storage Interface

This article explores the Container Storage Interface (CSI) and its role in integrating storage solutions with container orchestration platforms.

In this article, we explore the Container Storage Interface (CSI) and its critical role in modern container orchestration platforms. CSI provides a standardized way to integrate various storage solutions with container orchestrators, eliminating the need for bespoke, embedded storage code within the core of these systems.

Evolution in Container Orchestration

Historically, Kubernetes relied exclusively on Docker as the container runtime engine. All code for interfacing with Docker resided within the Kubernetes source. However, with the emergence of alternative container runtimes—such as Rocket and CRI-O—it became necessary to support multiple runtimes without altering Kubernetes' core codebase. This challenge led to the development of the Container Runtime Interface (CRI), a standard that enables orchestration tools like Kubernetes to communicate with different container runtimes seamlessly.

In a similar manner, the Container Networking Interface (CNI) was introduced to facilitate the integration of diverse networking solutions. CNI allows networking vendors to build plugins that adhere to standardized specifications, ensuring smooth operation within Kubernetes environments.

CSI extends these standardization principles to the storage domain, enabling support for multiple storage systems without modifications to container orchestration platforms.

The Role of CSI

The Container Storage Interface (CSI) empowers developers to create custom drivers for various storage systems. Popular examples include:

- Portworx
- Amazon EBS
- Azure Disk
- Dell EMC
- Isilon
- PowerMax

- Unity
- Extreme IO
- NetApp
- Nutanix
- HPE
- Hitachi
- Pure Storage

It is important to emphasize that CSI is not exclusive to Kubernetes. In fact, any container orchestration tool that implements the CSI standard can work with virtually any storage vendor supporting a CSI plugin. Currently, leading platforms such as Kubernetes, Cloud Foundry, and Mesos have adopted CSI.

How CSI Works

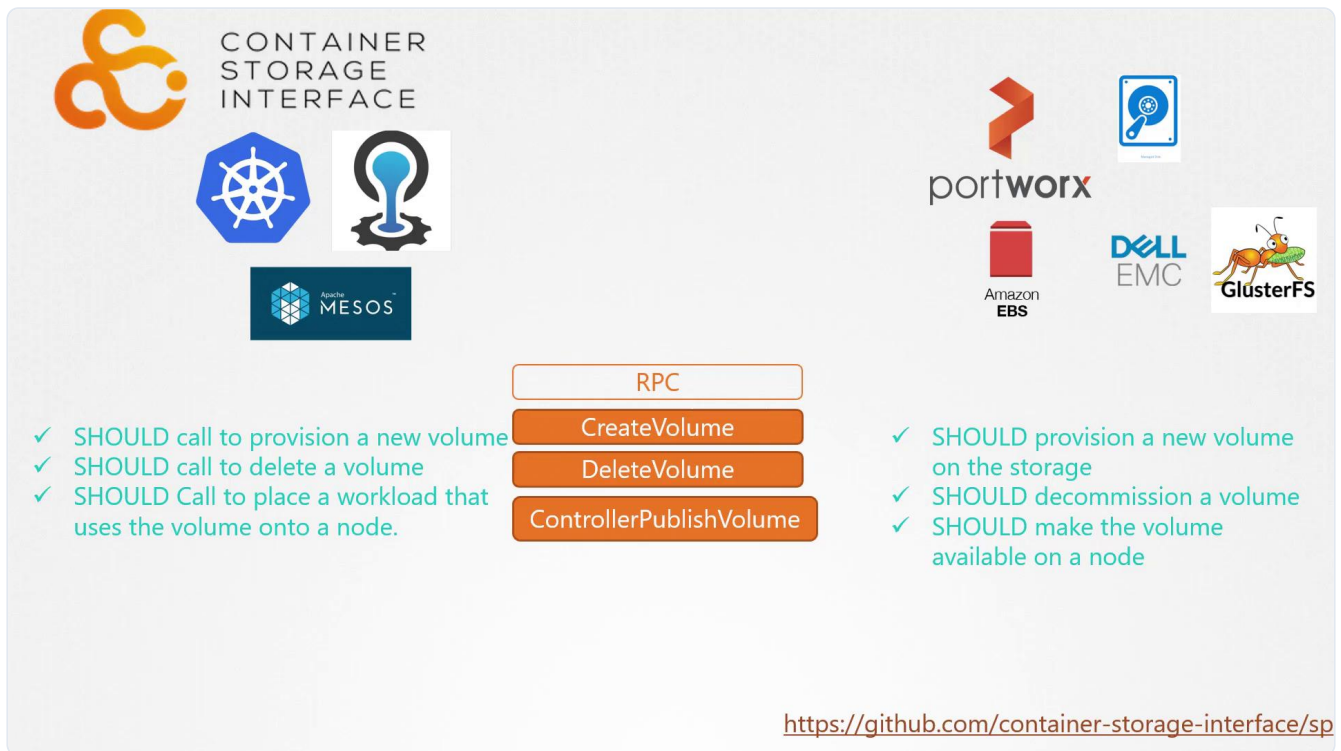
When a pod is created that requires persistent storage, the container orchestrator (e.g., Kubernetes) calls the "create volume" Remote Procedure Call (RPC) defined by the CSI standard. This call includes vital details such as the volume name and other parameters. The storage driver then processes the request by provisioning a new volume on the associated storage array and returns the result to the orchestrator.

Similarly, when a volume is no longer needed, the orchestrator issues a "delete volume" RPC. The storage driver responds by removing the specified volume from the storage system. The CSI specification outlines the required parameters, expected responses, and error codes for these operations, ensuring consistency and interoperability across different storage solutions.

For those seeking deeper technical insights, the full CSI specification is available on [GitHub](#).

CSI Architecture Diagram

Below is an illustrative diagram outlining the fundamental components of CSI. The image highlights a series of RPCs defined by the CSI standard, demonstrating how container orchestrators interact with storage drivers:



When integrating CSI, ensure that your storage drivers fully comply with the CSI specification to maintain seamless operation across diverse container orchestrators.

Conclusion

The Container Storage Interface represents a significant advancement in the way container orchestration platforms interact with storage systems. By standardizing communication through defined RPCs, CSI allows developers to integrate a wide range of storage vendors into their containerized environments with minimal friction.

This article provided an overview of CSI and its functionalities. In the next installment, we will delve into additional aspects of container orchestration, further enhancing your understanding of this dynamic ecosystem.

For more resources on container technologies, please refer to the following links:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [Docker Hub](#)
- [Terraform Registry](#)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Volumes

This lesson explores persistent volumes in Kubernetes, comparing them to Docker volumes and discussing various storage options with practical examples.

Hello, and welcome to this lesson on persistent volumes in Kubernetes. My name is Mumshad Mannambeth. In this lesson, we will start by exploring volumes in Kubernetes and comparing them to Docker volumes, then move on to a practical example and discuss different storage options.

Overview of Volumes in Docker

Docker containers are designed to be transient: they are created as needed, process data, and are then destroyed. As a consequence, any data stored within a container is lost when the container is removed. To overcome this limitation, Docker allows you to attach a volume to a container so that the data persists even after the container's lifecycle ends.

Volumes in Kubernetes

Similarly, Kubernetes pods are ephemeral by design. When a pod processes data and is subsequently deleted, the data inside it is normally lost. To retain this data, a volume is attached to the pod. Any data generated by the pod is stored in the volume and remains available even after the pod is terminated.

A Simple Example of Using Volumes

Consider a scenario where you have a single-node Kubernetes cluster. A pod generates a random number (between 0 and 100) and writes it to the file `/opt/number.out`. Without a volume, this file would be lost when the pod is deleted. By attaching a volume that uses a directory on the host for storage, the generated number is preserved.

In this example, the volume is configured to use the host's `/data` directory. This implies that any files created within the volume (for instance, the random number written to `/opt/number.out` in the container) are stored in `/data` on the node. The volume is then mounted to the `/opt` directory inside the container.

Below is the YAML configuration for the pod:

```
yaml theme={null} apiVersion: v1 kind: Pod metadata: name: random-number-generator spec:
containers: - image: alpine name: alpine command: ["/bin/sh", "-c"] args: ["shuf -i 0-100 -n 1
>> /opt/number.out;"] volumeMounts: - mountPath: /opt name: data-volume volumes: - name: data-
volume hostPath: path: /data type: Directory
```

When the pod executes the command, the random number is written to `/opt/number.out` inside the container. Since `/opt` is mounted to the host's `/data` directory, the file persists on the host even after the pod is deleted.

Volume Storage Options

In the above example, we utilized a `hostPath` volume, which is suitable for a single-node cluster. However, this approach is not ideal for multi-node clusters where each node's `/data` directory may differ. For such environments, Kubernetes supports several external and replicated storage solutions. Some popular storage options include:

- NFS
- GlusterFS
- Flocker
- Fibre Channel
- CephFS
- ScaleIO
- Public cloud storage solutions like AWS EBS, Azure Disk or File, and Google Persistent Disk

For example, to use an AWS Elastic Block Store (EBS) volume, replace the `hostPath` configuration with AWS EBS-specific parameters such as the volume ID and filesystem type. This allows your Kubernetes cluster to leverage AWS EBS for robust, scalable storage.

For a quick comparison, refer to the table below:

Storage Option	Use Case	Example Configuration
hostPath	Single-node clusters	Uses local directory
AWS EBS	Multi-node clusters on AWS	<code>awsElasticBlockStore</code>
Azure Disk	Multi-node clusters on Azure	<code>azureDisk</code>
Google Persistent Disk	Multi-node clusters on Google Cloud	<code>gcePersistentDisk</code>
NFS	Shared storage across nodes	NFS volume configuration

Conclusion

This lesson provided an overview of volumes in Kubernetes by comparing them with Docker volumes and demonstrating how to use a `hostPath` volume for data persistence. In our next lesson, we will explore persistent volumes in Kubernetes, which offer advanced storage management and additional capabilities tailored for multi-node clusters.

For further reading, check out these resources:

- [Kubernetes Basics](#)
- [Kubernetes Documentation](#)
- [AWS EBS Documentation](#)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Persistent Volumes

This lesson explores centralizing storage management in Kubernetes using persistent volumes and how to create and manage them effectively.

Hello and welcome to this lesson on persistent volumes. I'm Mumshad Mannambeth, and in this session, we'll explore how to centralize storage management in Kubernetes using persistent volumes.

Previously, we discussed how volumes are defined within pod manifest files, where the storage settings are directly included in each pod definition. For example, a typical pod configuration with a volume might look like:

```
```yaml theme={null} volumes: - name: data-volume awsElasticBlockStore: volumeID: fsType: ext4
```

In environments where many users deploy multiple pods, duplicating storage configuration in every pod file is not ideal.

To solve this issue, administrators can create a centralized pool of storage. Users then request portions of this storage for their pods.

A persistent volume is a cluster-wide storage resource defined and managed by an administrator. Applications can then request a portion of this storage for their pods.

<Frame>  
! [The image illustrates the relationship between Persistent Volume Claims (PVCs) and Persistent Volumes]  
</Frame>

### ## Creating a Persistent Volume

In this section, we will create a persistent volume using a base template. First, update the API version.

- \* ReadWriteOnce: The volume can be mounted as read-write by a single node.
- \* ReadOnlyMany: The volume can be mounted as read-only by multiple nodes.
- \* ReadWriteMany: The volume can be mounted as read-write by multiple nodes.

Below is an initial portion of the configuration that defines the persistent volume along with its access modes.

```
```yaml theme={null}
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-vol1
spec:
  accessModes:
    - ReadWriteOnce
```

Next, specify the storage capacity for this persistent volume. In our example, we set the capacity to 1Gi. After defining the capacity, choose the volume type. Here, we use the `hostPath` option, which leverages storage from the node's local directory.

The `hostPath` option is primarily for testing or single-node setups and is not recommended for production environments.

The complete persistent volume manifest appears as follows:

```
```yaml theme={null} apiVersion: v1 kind: PersistentVolume metadata: name: pv-vol1 spec: accessModes: - ReadWriteOnce capacity: storage: 1Gi hostPath: path: /tmp/data
```

To create the persistent volume, execute the following command:

```
```bash theme={null}
kubectl create -f pv-d
```

After creating the volume, you can verify its status by running:

```
bash theme={null} kubectl get persistentvolume
```

In a production setup, replace the `hostPath` option with a supported storage solution, such as AWS Elastic Block Store, to ensure data durability and scalability.

This concludes our discussion on persistent volumes. Up next, we will explore persistent volume claims (PVCs) to see how you can request storage from the centralized persistent volume pool.

For further reading, check out these helpful resources:

- [Kubernetes Documentation](#)
- [Kubernetes Basics](#)
- [AWS Elastic Block Store](#)
- [Docker Hub](#)

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

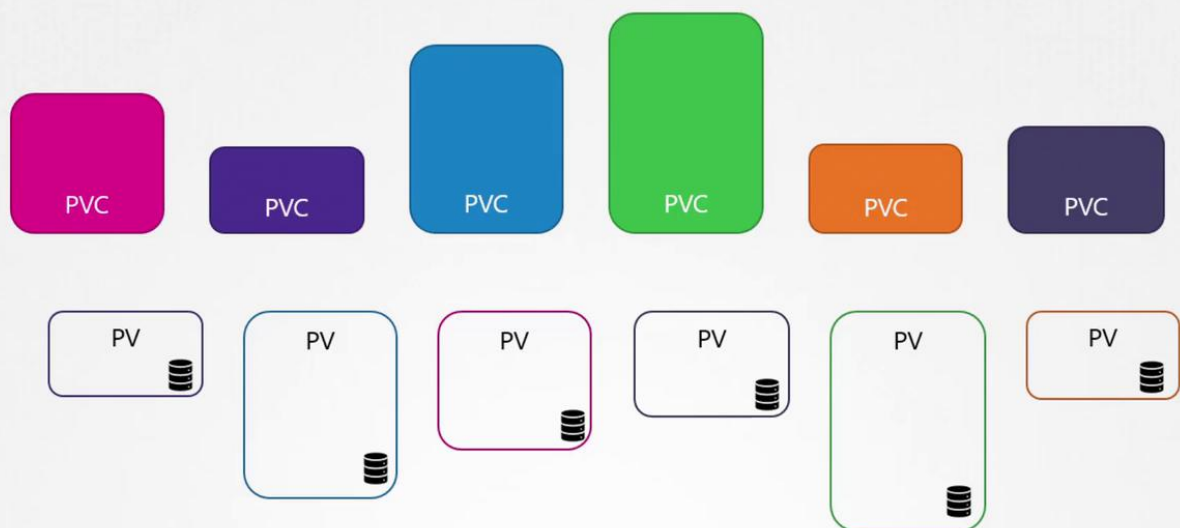
Persistent Volume Claims

This article explains how to create and manage Persistent Volume Claims in Kubernetes, including binding, deletion, and reclaim policies.

Welcome to this article on Persistent Volume Claims (PVCs) in Kubernetes. In our previous lesson, we created a Persistent Volume (PV). Now, we will explore how to create a Persistent Volume Claim (PVC) to expose that storage to a node.

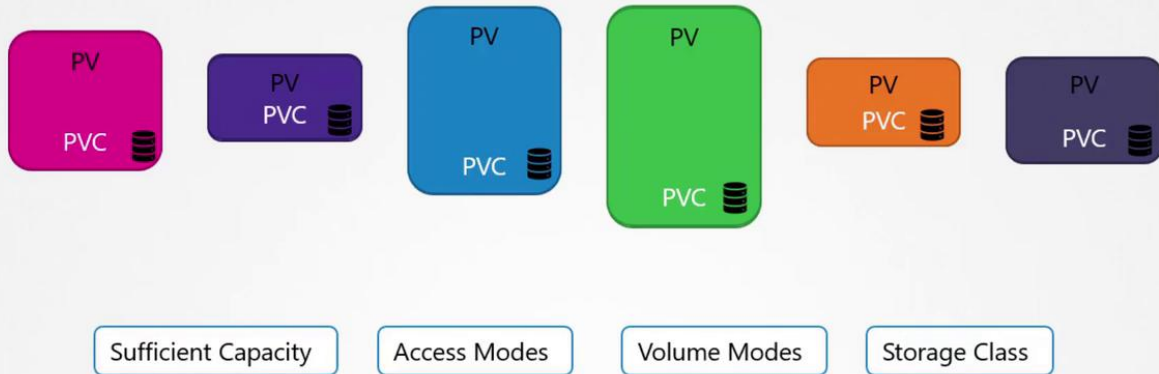
Persistent volumes and persistent volume claims are two distinct objects in Kubernetes. An administrator is responsible for creating PVs, while users create PVCs to request storage resources. When a PVC is created, Kubernetes automatically binds it to a PV that meets the requested capacity, access modes, volume modes, and storage class.

Persistent Volume Claim



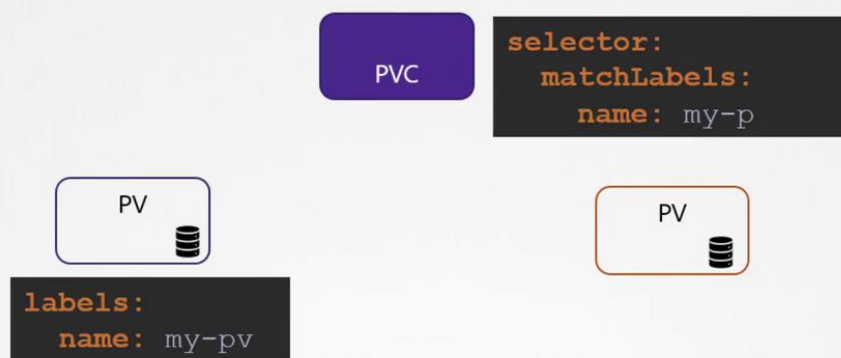
Kubernetes evaluates several factors when binding a PVC to a PV. If multiple PVs can satisfy a claim, you can use labels and selectors to bind the claim to a specific volume.

Binding



It is important to note that if a smaller PVC is matched with a larger PV that meets all criteria, the unrequested capacity remains unused by any other PVC. If no PV satisfies the claim's requirements, the PVC will remain in a pending state until a new, suitable PV becomes available.

Binding



- Sufficient Capacity
- Access Modes
- Volume Modes
- Storage Class
- Selector

Creating a Persistent Volume Claim

Below is an example YAML template for creating a PVC. In this configuration, we set the API version to v1 with kind PersistentVolumeClaim, and name it "myclaim". Under the specification section, the access mode is set to ReadWriteOnce, and 500 MiB of storage is requested.

```
```yaml theme={null} apiVersion: v1 kind: PersistentVolumeClaim metadata: name: myclaim spec: accessModes: - ReadWriteOnce resources: requests: storage: 500Mi
```

To create the PVC:

1. Save the above YAML to a file, for example, `pvc-definition.yaml`.
2. Run the command below in your terminal:

```
```bash theme={null}
kubectl create -f pvc-definition.yaml
```
```

You can verify the created PVC by executing:

```
```bash theme={null}
kubectl get persistentvolumeclaim
NAME      STATUS   VOLUME   CAPACITY   ACCESS MODES
myclaim   Pending
```

Kubernetes will inspect the available PV. Suppose, in our example, a PV is configured with 1GiB storage and compatible access modes — if it meets the PVC's criteria, it will automatically bind to the PVC. Here is an example of such a PV definition:

```
```yaml theme={null} apiVersion: v1 kind: PersistentVolume metadata: name: pv-voll spec: accessModes: - ReadWriteOnce capacity: storage: 1Gi awsElasticBlockStore: volumeID: fsType: ext4
```

After the binding process, running the `kubectl get persistentvolumeclaim` command will show that the PVC

## Deleting a PVC and Persistent Volume Reclaim Policies

To delete a PVC, use the following command:

```
```bash theme={null}
kubectl delete persistentvolumeclaim myclaim
```

When a PVC is deleted, what happens next depends on the underlying persistent volume's reclaim policy. The reclaim policy determines the fate of the PV and can be configured as follows:

Reclaim Policy	Description
Retain	The PV remains in the cluster after the PVC is deleted. An administrator must manually reclaim it.
Delete	The PV is automatically deleted along with the PVC, releasing the storage on the physical device.
Recycle	The PV data is scrubbed before reuse by new claims.

The "Recycle" reclaim policy is deprecated in recent Kubernetes versions and might not be available in your cluster.

For example, to set the reclaim policy to Retain, you would include:

```
yaml theme={null} persistentVolumeReclaimPolicy: Retain
```

Choose the reclaim policy that best fits your storage management strategy.

That concludes our guide on Persistent Volume Claims. We encourage you to practice configuring, binding, and troubleshooting PVs and PVCs in your Kubernetes environment. For further reading, check out the [Kubernetes Documentation](#) and other related resources.

Documentation Index

Fetch the complete documentation index at: <https://notes.kodekloud.com/lms.txt> Use this file to discover all available pages before exploring further.

Storage Class

This article explores Kubernetes storage classes and their role in simplifying storage provisioning for applications.

In this lesson, we explore storage classes in Kubernetes and demonstrate how they simplify the process of storage provisioning for applications. Traditionally, administrators manually created PersistentVolumes (PVs) and PersistentVolumeClaims (PVCs) and mounted them to pods. This guide covers both static provisioning (manually creating disks and PVs) and dynamic provisioning using storage classes, making your Kubernetes storage management more efficient.

Static Provisioning

With static provisioning, you manually create the underlying storage (for example, a [Google Cloud persistent disk](#)) and then construct a PV that references that disk. Each time an application requires storage, you must provision a disk on Google Cloud and create the corresponding PV definition.

For example, to create a persistent disk on Google Cloud, you can use the following command:

```
```bash theme={null} gcloud beta compute disks create \ --size 1GB \ --region us-east1 \ pd-disk
```

Then, define your Kubernetes resources as follows:

```
```yaml theme={null}
# pv-definition.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-vol1
spec:
  accessModes:
    - ReadWriteOnce
  capacity:
    storage: 500Mi
  gcePersistentDisk:
    pdName: pd-disk
    fsType: ext4
```

```
```yaml theme={null}
```

# pvc-definition.yaml

---

```
apiVersion: v1 kind: PersistentVolumeClaim metadata: name: myclaim spec: accessModes: - ReadWriteOnce
resources: requests: storage: 500Mi
```

```
```yaml theme={null}
# pod-definition.yaml
apiVersion: v1
kind: Pod
metadata:
  name: random-number-generator
spec:
  containers:
    - image: alpine
      name: alpine
      command: ["/bin/sh", "-c"]
      args: ["shuf -i 0-100 -n 1 >> /opt/number.out;"]
      volumeMounts:
        - mountPath: /opt
          name: data-volume
  volumes:
    - name: data-volume
      persistentVolumeClaim:
        claimName: myclaim
```

In this setup, the PVC binds to the manually created PV that refers to your existing Google Cloud persistent disk.

Dynamic Provisioning with Storage Classes

Dynamic provisioning removes the need for manual storage pre-provisioning. When you create a PVC, the associated storage class automatically provisions the necessary PV using the defined provisioner.

First, create a storage class object that specifies the provisioner (in this case, Google Cloud's persistent disk):

```
```yaml theme={null}
```

# sc-definition.yaml

---

```
apiVersion: storage.k8s.io/v1 kind: StorageClass metadata: name: google-storage provisioner: kubernetes.io/
gce-pd
```

With the storage class in place, update your PVC to reference it for dynamic provisioning:

```
```yaml theme={null}
# pvc-definition.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: myclaim
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: google-storage
  resources:
    requests:
      storage: 500Mi
```

The pod definition remains similar:

```
```yaml theme={null}
```

## pod-definition.yaml

```
apiVersion: v1 kind: Pod metadata: name: random-number-generator spec: containers: - image: alpine name:
alpine command: ["/bin/sh", "-c"] args: ["shuf -i 0-100 -n 1 >> /opt/"] volumeMounts: - mountPath: /opt name:
data-volume volumes: - name: data-volume persistentVolumeClaim: claimName: myclaim
```

When you create a PVC with a storage class specified, Kubernetes leverages the defined provisioner to dynamically provision storage.

<Callout icon="lightbulb" color="#1CB2FE">

Using dynamic provisioning simplifies storage management by reducing manual tasks and minimizing potential for human error.

</Callout>

\*\*\*

### ## Customizing Storage Classes

Storage classes in Kubernetes support various parameters, allowing you to fine-tune the provisioned storage.

Below are examples of customized storage class definitions:

```
```yaml theme={null}
# silver storage class: standard disk without replication
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: silver
provisioner: kubernetes.io/gce-pd
parameters:
  type: pd-standard
  replication-type: none
```

```
```yaml theme={null}
```

# gold storage class: SSD disk without replication

---

```
apiVersion: storage.k8s.io/v1 kind: StorageClass metadata: name: gold provisioner: kubernetes.io/gce-pd
parameters: type: pd-ssd replication-type: none
```

```
```yaml theme={null}
# platinum storage class: SSD disk with regional replication
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: platinum
provisioner: kubernetes.io/gce-pd
parameters:
  type: pd-ssd
  replication-type: regional-pd
```

By specifying the appropriate storage class in your PVC definitions, you match the storage's performance and reliability to your application's needs.

Summary

This lesson demonstrated how Kubernetes storage classes streamline persistent storage provisioning. Whether using static provisioning or dynamic provisioning, storage classes allow for more efficient management of storage resources, reducing manual efforts and enhancing scalability.

For further details on Kubernetes storage and dynamic provisioning, visit the [Kubernetes Documentation](#).